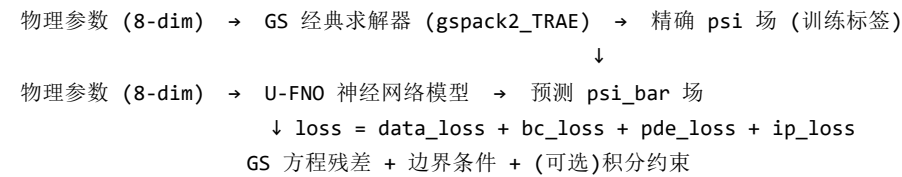


GS_PINO 技术文档

1. 项目概述

GS_PINO 是一个基于**神经网络算子**的固定边界 Grad-Shafranov (GS) 方程求解器。项目使用 **U-FNO** (U-Net + Fourier Neural Operator) 架构，学习从 8 个物理参数到归一化极向磁通 ψ_{bar} 的映射。

核心流程



文件结构

src/gs_pino/	
├─ __init__.py	# 包初始化
├─ geometry.py	# LCFS 几何工具: Miller 参数化、掩码、SDF
├─ solvers.py	# 固定边界 GS 求解器适配器: gspack2_TRAE 封装 + 解析备用
├─ generate_dataset.py	# 固定边界数据集生成 CLI
├─ data.py	# 固定边界数据集类 GSDataset
├─ models.py	# 模型定义: UFN02d/UFN02d_v2、PlaNetCore、PlaNetAttention 等
├─ losses.py	# 固定边界损失函数 (masked MSE, GS 残差, axis, Ip/betap 约束)
├─ train.py	# 固定边界训练 CLI
├─ evaluate.py	# 固定边界评估 CLI
├─ generate_freegs_dataset.py	# 自由边界数据集生成 CLI (freegs)
├─ data_freegs.py	# 自由边界数据集类 FreeBndDataset + U-FNO 输入构建
├─ losses_freegs.py	# 自由边界损失函数 (GSOperatorLoss, curvature, axis, ip)
├─ train_freegs.py	# 自由边界训练 CLI (--model planet/ufno)
├─ evaluate_freegs.py	# 自由边界评估 CLI
├─ visualize_freegs.py	# 自由边界可视化 CLI
├─ inference_freegs.py	# 自由边界推理 CLI (见 14.2)
├─ merge_datasets.py	# 数据集合并工具
├─ visualize_results.py	# 结果可视化 (见 14.3)
└─ verification/	# Solov'ev 解析解与 gspack 求解器验证脚本 (第 13 章)

2. 物理背景：Grad-Shafranov 方程

托卡马克等离子体平衡由 Grad-Shafranov 方程描述：

$$\Delta^* \psi + \mu_0 R^2 p'(\psi) + FF'(\psi) = 0$$

其中：

- ψ — 极向磁通函数 (Poloidal magnetic flux)
- Δ^* — 修正 Laplace 算子: $\Delta^* \psi = \partial^2 \psi / \partial R^2 - (1/R) \cdot \partial \psi / \partial R + \partial^2 \psi / \partial Z^2$
- $p(\psi)$ — 等离子体压强剖面
- $F(\psi) = R \cdot B_\phi$ — 极向电流函数
- $\mu_0 = 4\pi \times 10^{-7}$ — 真空磁导率

剖面模型 (Jeon 2015)

使用 `ConstrainBetapIp` 参数化电流和压强剖面：

$$p'(\psi_N) = L \cdot \text{Beta0} / R_{\text{axis}} \cdot (1 - \psi_N^{\alpha_m})^{\alpha_n}$$

$$FF'(\psi_N) = \mu_0 \cdot L \cdot (1 - \text{Beta0}) \cdot R_{\text{axis}} \cdot (1 - \psi_N^{\alpha_m})^{\alpha_n}$$

其中 $\psi_N = (\psi - \psi_{\text{lcfs}}) / (\psi_{\text{axis}} - \psi_{\text{lcfs}})$ 是归一化磁通，LCFS 处为 0，磁轴处为 1。

3. 数据生成流程

3.1 参数采样 (`generate_dataset.py::sample_params`)

```
ranges = {
    "R0":      (0.8, 1.5),      # 主半径 [m]
    "a":       (0.3, 0.7),      # 小半径 [m]
    "kappa":   (1.0, 2.0),      # 拉长比
    "delta":   (0.0, 0.5),      # 三角形变
    "Ip":      (1e5, 5e5),      # 等离子体电流 [A]
    "betap":   (0.3, 1.5),      # 极向比压
    "alpha_m": (0.5, 3.0),      # 电流剖面指数 1
    "alpha_n": (0.5, 3.0),      # 电流剖面指数 2
}
```

3.2 经典求解器调用 (`solvers.py::GSSolverAdapter`)

经典求解器来自 [GS_solver](#) (本项目中以 `gspack2_TRAE` 目录引用)，使用有限差分法 + Picard 迭代求解固定边界 GS 方程。

```
物理参数 (dict)
↓
FixedBoundaryEquilibrium(R0, a, kappa, delta, fix_bndry_zero=True, nx=nr_solve, ny=nz_solve)
↓ + ConstrainBetapIp(betap, Ip, alpha_m, alpha_n, Raxis=R0, fvac=1.0)
↓
picard.solve(eq, pro, maxits=50, rtol=rtol, anderson_m=5)
↓
返回: {R, Z, psi, psi_bar, psi_lcfs=0, psi_axis, R_axis, Z_axis,
       plasma_mask, profile_params=[L, Beta0]}
```

`fix_bndry_zero=True` 使求解器直接返回 LCFS 处 $\psi=0$ 的结果，省去后续平移步骤，同时求解速度提升约 1.6 倍。

求解网格与插值：求解器在最近的 $2^k + 1$ (Romberg) 网格上迭代 (`nx=nr_solve`, `ny=nz_solve`)，收敛后用 `RectBivariateSpline` 双三次样条插值回目标网格 (`nr`, `nz`)。因此任意目标网格 (如 64×64) 均可直接使用， 129×129 恰好是 2^7+1 时无需插值。

求解失败过滤：求解结果若满足以下任一条件则视为失败并返回 `None`，数据集生成时跳过该样本：

- `psi_bar` 越界 (`< -0.01` 或 `> 1.01`)
- `|psi_axis| > 1e5`
- `|L| > 1e8` 或 `|Beta0| > 1e3`

收敛容差：`GSolverAdapter` 内部默认 `rtol=5e-3`；`generate_dataset` 的 `--rtol` (默认 `1e-5`) 会覆盖该值。

3.3 几何通道 (geometry.py)

每个样本在保存时包含 4 个几何通道：

通道	含义	计算方式
mask	LCFS 内部二值掩码	求解器返回 <code>plasma_mask</code> 或 $\rho \leq 1.0$
sdf	符号距离函数 (近似)	$\rho - 1.0$ ，内部为负
rho	归一化磁面半径	Miller 近似
theta	极向角	$\text{atan2}(Z/(\kappa a), (R-R_0)/a)$

3.4 保存格式 (.npz)

```
data/gs_large.npz
├─ R, Z           [n, nr, nz]   - 计算网格坐标
├─ params         [n, 8]       - 8 个物理参数
├─ psi            [n, nr, nz]   - 真实 psi (LCFS=0)
├─ psi_bar        [n, nr, nz]   - 归一化 psi (LCFS=0, axis=1)
├─ mask           [n, nr, nz]   - 等离子体掩码
├─ sdf            [n, nr, nz]   - 符号距离函数
├─ rho            [n, nr, nz]   - 归一化半径
├─ theta          [n, nr, nz]   - 极向角
├─ axes           [n, 4]       - [R_axis, Z_axis, psi_lcfs, psi_axis]
├─ profile_params [n, 2]       - [L, Beta0]
└─ param_names    [8]         - 参数名称
```

4. U-FNO 模型架构

4.1 输入通道构建 (data.py::build_input)

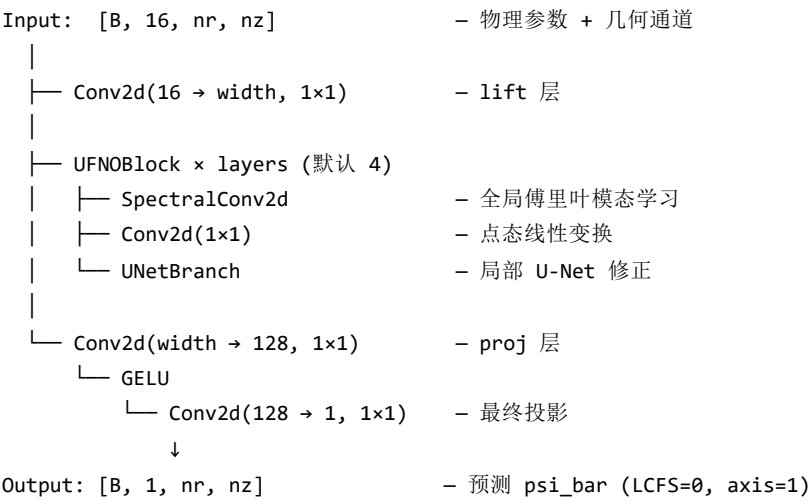
输入张量形状: [C, nr, nz] , 共 **8 + 8 = 16 通道**:

```
8 个几何/坐标通道:
[(R-R0)/a, Z/a, Z/(kappa*a), mask, sdf, rho, sin(theta), cos(theta)]

8 个物理参数通道 (归一化后广播到全网格):
[R0_norm, a_norm, kappa_norm, delta_norm, Ip_norm, betap_norm, alpha_m_norm, alpha_n_norm]
```

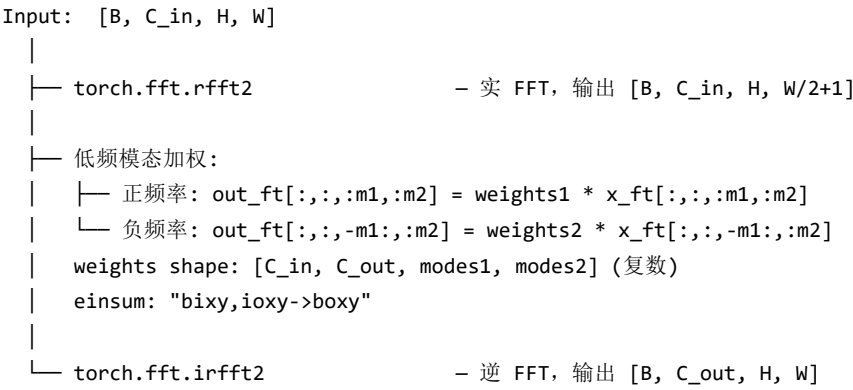
归一化使用训练集的均值和标准差: `p_norm = (p - mean) / std`。

4.2 整体架构



4.3 各模块详细结构

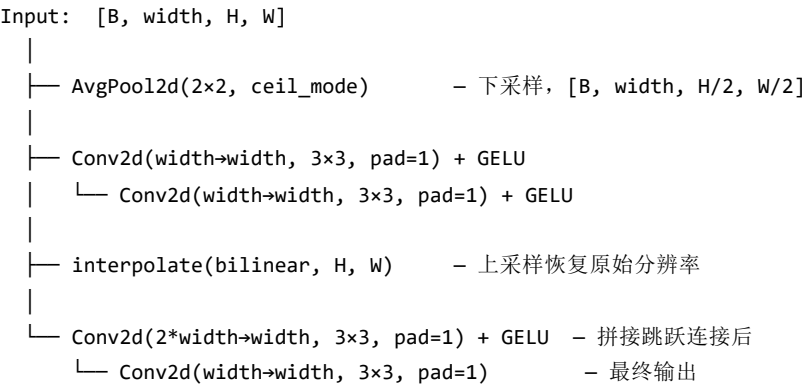
4.3.1 SpectralConv2d (傅里叶谱卷积)



关键参数:

- `modes1`：R 方向保留的傅里叶模态数（UFNO2d 类默认 16，UFNO2d_v2 默认 32）
- `modes2`：Z 方向保留的傅里叶模态数（同上）
- 模态数会自动裁剪为不超过网格尺寸的一半

4.3.2 UNetBranch (局部修正分支)



4.3.3 UFNOBlock (完整块)

$$\text{Output} = \text{GELU}(\text{SpectralConv2d}(x) + \text{PointwiseConv2d}(x) + \text{UNetBranch}(x))$$

三个分支相加后经过 GELU 激活：

- **Spectral path**：全局傅里叶模态学习
- **Pointwise path**：逐点通道混合（1×1 卷积）
- **UNet path**：局部特征提取和下采样/上采样融合

4.4 完整张量形状流程示例

以 `batch=16, nr=129, nz=129, width=64, layers=4, modes1=32, modes2=32` 为例：

层	输出形状
Input	[16, 16, 129, 129]
lift (Conv2d 1×1)	[16, 64, 129, 129]
UFNOBlock 1	[16, 64, 129, 129]
├─ SpectralConv2d	[16, 64, 129, 129]
├─ rfft2	[16, 64, 129, 65]
├─ modal multiply	[16, 64, 129, 65]
└─ irfft2	[16, 64, 129, 129]
├─ Conv2d(1×1)	[16, 64, 129, 129]
└─ UNetBranch	[16, 64, 129, 129]
├─ AvgPool2d	[16, 64, 65, 65] (ceil_mode)
├─ Conv2d(3×3)×2	[16, 64, 65, 65]
├─ interpolate	[16, 64, 129, 129]
└─ Conv2d(3×3)×2	[16, 64, 129, 129]
UFNOBlock 2-4 (同结构)	[16, 64, 129, 129]
proj (Conv2d 1×1→128→1)	[16, 1, 129, 129]

4.5 U-FNO v2 (UFN02d_v2)

在 v1 基础上改进的增强版本，主要差异：

```
UFNOBlock_v2:
    residual = x
    x = GroupNorm(4, width)(x)                # 归一化
    x = GELU(SpectralConv2d(x) + PointwiseConv2d(x) + UNetBranch(x))
    return x + residual                        # 残差连接
```

- **GroupNorm**：替代无归一化，训练更稳定
- **残差连接**：缓解深层网络梯度消失
- **默认超参数更大**：width=128, layers=6, modes1=modes2=32 (v1 为 32/4/16)，proj 中间层 256
- 该变体主要服务于自由边界管线 (train_freegs --model ufno ，12 通道输入)，也可用于固定边界任务

5. 损失函数 (losses.py)

5.1 数据损失: masked_mse

仅在 LCFS 内部区域计算均方误差：

```
loss_data =  $\Sigma[(pred - target)^2 \cdot mask] / \Sigma[mask]$ 
```

5.2 边界条件损失: boundary_band_loss

在 LCFS 附近 (|sdf| < 0.04) 惩罚非零值，强制边界条件 $\psi_{bar}|_{LCFS} = 0$ ：

```
band = (|sdf| < width).float()          # width = 0.04
loss_bc =  $\Sigma[pred^2 \cdot band] / \Sigma[band]$ 
```

5.3 GS 方程残差: gs_residual_loss

计算 $\Delta\psi_{bar}$ 并与源项比较：

步骤 1: 计算 $\Delta\psi_{bar}$ (有限差分)

```
 $\partial^2\psi/\partial R^2 \approx (\psi[i+1,j] - 2*\psi[i,j] + \psi[i-1,j]) / dR^2$ 
 $\partial\psi/\partial R \approx (\psi[i+1,j] - \psi[i-1,j]) / (2*dR)$ 
 $\partial^2\psi/\partial Z^2 \approx (\psi[i,j+1] - 2*\psi[i,j] + \psi[i,j-1]) / dZ^2$ 
```

```
 $\Delta\psi_{bar} = \partial^2\psi/\partial R^2 - (1/R) \cdot \partial\psi/\partial R + \partial^2\psi/\partial Z^2$ 
```

步骤 2: 计算源项

```
psiN = 1.0 - psi_bar # 归一化磁通 [0,1]
shape = (1 - psiN^{am})^{an} # 剖面形状
S = \mu_0 \cdot L \cdot [\text{Beta0} \cdot R^2 / R0 + (1 - \text{Beta0}) \cdot R0] \cdot \text{shape}
```

步骤 3: 残差

```
dpsi = psi_axis - psi_lcfs # 归一化因子
residual = lap_psi_bar + S / dpsi
loss_pde = \sum[\text{residual}^2 \cdot \text{mask}] / \sum[\text{mask}]
```

5.4 Ip 约束损失 (可选): ip_constraint_loss

通过积分 $\int \mathbf{j} \cdot \mathbf{\phi}$ 计算预测 I_p 并与目标比较:

```
J_phi = L \cdot [\text{Beta0} \cdot R / R0 + (1 - \text{Beta0}) \cdot R0 / R] \cdot \text{shape}
Ip_pred = \iint J_phi \, dR \, dZ
loss_ip = (Ip_pred - Ip_target)^2 / Ip_target^2
```

5.5 betap 约束损失 (可选): betap_constraint_loss

```
\beta p = 2\mu_0 \cdot \iint p \cdot R \, dR \, dZ / \iint B_{pol}^2 \cdot R \, dR \, dZ

p(\psi_N) = dpsi \cdot L \cdot \text{Beta0} / R0 \cdot \int_0^{1-\psi_N} (1-s^{am})^{an} \, ds # 数值积分
B_{pol}^2 = B_r^2 + B_z^2
B_r = -(1/R) \cdot \partial\psi / \partial Z
B_z = (1/R) \cdot \partial\psi / \partial R
```

5.6 磁轴约束损失: axis_constraint_loss

在真实磁轴位置（由元数据中的 R_{axis} , Z_{axis} 双线性插值定位）惩罚预测值偏离 1，强制归一化磁通在磁轴处为 1（ ψ_{bar} 在磁轴处的目标值）:

```
# 在 (R_axis, Z_axis) 处双线性插值预测场
i_frac = (R_axis - R[:, 0, 0]) / dR
j_frac = (Z_axis - Z[:, 0, 0]) / dZ
axis_pred = 双线性插值(pred, i_frac, j_frac)
loss_axis = (axis_pred.clamp(-5, 5) - 1.0)^2 # 均值
```

5.7 总损失

```
loss = loss_data + bc_weight * loss_bc + pde_weight * loss_pde
      + ip_weight * loss_ip + axis_weight * loss_axis
```

默认权重 ([train.py CLI](#)): $bc=0.05$, $pde=0.01$, $ip=0.0$, $axis=0.01$

注: `betap_constraint_loss` (5.5 节) 定义在 `losses.py` 中但未接入任何训练脚本, 没有对应 CLI 参数; 5.4 节的 `ip_constraint_loss` 同样默认关闭 (`--ip-weight 0.0`)。

6. 训练流程 (train.py)

6.1 数据划分

总样本 n

- └─ 测试集: $n * 0.15$ (随机排列后取最前 15%)
- └─ 验证集: $n * 0.15$ (中间 15%)
- └─ 训练集: $n * 0.70$ (最后 70%)

注意：测试集取排列**最前** 15%，训练集取**最后** 70% (`data.py::split_indices`)。

6.2 每 epoch 步骤

```
for x, y, mask, sdf, params, meta_list in loader:
    meta = stack_metadata(meta_list)      # 合并元数据为 batch tensor

    pred = model(x)

    loss_data = masked_mse(pred, y, mask)
    loss_bc   = bc_weight * boundary_band_loss(pred, sdf)
    loss_pde  = pde_weight * gs_residual_loss(pred, ..., meta)
    loss_ip   = ip_weight * ip_constraint_loss(pred, ..., meta)
    loss_axis = axis_weight * axis_constraint_loss(pred, ..., meta)

    loss = loss_data + loss_bc + loss_pde + loss_ip + loss_axis

    if training:
        loss.backward()
        optimizer.step()
```

损失在梯度累积 (`--accum-steps`) 下按 `loss / accum_steps` 反传，每 `accum_steps` 个 batch 执行一次 `optimizer.step()`；启用 `--amp` 时使用混合精度 (`GradScaler`)。

6.3 Checkpoint 保存

当验证损失最低时保存 `best.pt`：

```
torch.save({
    "model": model.state_dict(),
    "args": vars(args),           # 命令行参数
    "param_mean": train_ds.param_norm.mean,
    "param_std": train_ds.param_norm.std,
    "test_indices": test_idx,     # 测试集索引
}, "best.pt")
```

6.4 训练监控

每个 epoch 记录到 `history.json`：

- train_data, train_bc, train_pde, train_ip, train_axis, train_total
- val_data, val_bc, val_pde, val_ip, val_axis, val_total
- epoch, lr

自动生成 training_curves.png 展示各损失变化曲线。

7. 评估流程 (evaluate.py)

7.1 指标

```
{
  "masked_mse": 0.00014,           // 测试集 masked MSE
  "relative_l2_mean": 0.023,       // 平均相对 L2 误差
  "relative_l2_median": 0.016,     // 中位数
  "relative_l2_p95": 0.065,        // 95 分位数
  "relative_l2_max": 0.19,         // 最大值
  "n_cases": 120                  // 测试样本数
}
```

7.2 可视化

每个测试样本生成三面板对比图：

1. **true psi** — 经典求解器输出的真实 psi 场
2. **pred psi** — 模型预测的 psi 场 (psi_bar 还原为真实 psi)
3. **pred - true** — 预测误差

整体统计图：

- summary_error_histogram.png — 误差分布直方图
- summary_error_vs_parameters.png — 误差与各输入参数的关系散点图

最优/最差样本：额外将相对 L2 误差最小的 3 个与最大的 3 个样本分别保存到 best_cases/ 与 worst_cases/ 目录。

8. 配置文件与命令行参数

8.1 generate_dataset

参数	默认值	说明
--out	data/gs_fixed_boundary.npz	输出路径
--n-samples	64	样本数
--nr	64	R 网格数
--nz	64	Z 网格数

参数	默认值	说明
--seed	42	随机种子
--rtol	1e-5	求解器收敛容差
--n-jobs	-1	joblib 并行生成线程数（-1 = 全部核心）

8.2 train

参数	默认值	说明
--data	data/gs_fixed_boundary.npz	数据集路径
--output-dir	outputs/run	输出目录
--epochs	20	训练轮数
--batch-size	4	批次大小
--lr	1e-3	学习率
--width	32	模型通道宽度
--modes1	16	R 方向傅里叶模态数
--modes2	16	Z 方向傅里叶模态数
--layers	4	U-FNO 块数量
--pde-weight	0.01	PDE 残差损失权重
--bc-weight	0.05	边界损失权重
--ip-weight	0.0	lp 约束损失权重
--axis-weight	0.01	磁轴约束损失权重
--clip-grad	1.0	梯度裁剪阈值 (0=禁用)
--accum-steps	2	梯度累积步数（有效 batch = batch-size × accum-steps）
--amp	False	混合精度训练
--warmup-epochs	5	学习率线性预热轮数
--patience	30	早停耐心值（0=禁用）
--min-epochs	50	早停前最小训练轮数
--seed	0	随机种子

8.3 evaluate

参数	默认值	说明
--data	data/gs_fixed_boundary.npz	数据集路径
--checkpoint	必填	模型权重文件

参数	默认值	说明
--output-dir	outputs/eval	输出目录
--batch-size	4	评估批次大小
--max-plots	12	最大可视化图数

9. 运行示例

模块调用方式：项目为 src 布局（pyproject.toml），python -m gs_pino.* 需要先 pip install -e . 或设置 PYTHONPATH=src。仓库内的 scripts/run_smoke.sh、scripts/run_practical.sh 使用 export PYTHONPATH=...:src 方式（见 14.5 节）。

9.1 端到端流程

```
# 1. 生成数据集（129×129 网格，1000 样本）
python -m gs_pino.generate_dataset --out data/gs_large.npz \
    --n-samples 1000 --nr 129 --nz 129 --seed 42 --rtol 1e-5

# 2. 训练模型
python -m gs_pino.train --data data/gs_large.npz \
    --epochs 200 --batch-size 16 --width 64 --modes1 32 --modes2 32 \
    --layers 4 --pde-weight 0.01 \
    --output-dir outputs/large

# 3. 评估模型
python -m gs_pino.evaluate --data data/gs_large.npz \
    --checkpoint outputs/large/best.pt \
    --output-dir outputs/large_eval --max-plots 10
```

9.2 快速测试

```
# 小数据集快速验证
python -m gs_pino.generate_dataset --out data/gs_smoke.npz \
    --n-samples 20 --nr 65 --nz 65 --seed 42

python -m gs_pino.train --data data/gs_smoke.npz \
    --epochs 10 --batch-size 4 --width 16 --modes1 8 --modes2 8 \
    --layers 2 --pde-weight 0.01 --output-dir outputs/smoke

python -m gs_pino.evaluate --data data/gs_smoke.npz \
    --checkpoint outputs/smoke/best.pt \
    --output-dir outputs/smoke_eval --max-plots 3
```

10. 参考结果

Large 配置（984 训练样本，129×129 网格，200 epochs）

版本 1 — 基线 (旧)

指标	值
最佳验证损失	0.00085
测试 masked MSE	0.00014
相对 L2 均值	2.32%
相对 L2 中位数	1.62%
相对 L2 P95	6.54%
相对 L2 最大值	19.3%
训练稳定性	epoch ~150 发散后恢复

版本 2 — 余弦退火 + 梯度裁剪 (改进)

指标	值	提升
最佳验证损失	0.00033	2.6×
测试 masked MSE	0.000028	5.1×
相对 L2 均值	1.09%	2.1×
相对 L2 中位数	0.93%	1.7×
相对 L2 P95	1.73%	3.8×
相对 L2 最大值	7.97%	2.4×
训练稳定性	全程稳定下降	✓

版本 3 — 数据翻倍 + 低 LR + 梯度累积 (2024)

数据集扩大到 1960 样本（由 2000 样本过滤得到，seed=42）。由于样本量增大，需要降低学习率防止梯度爆炸。同时使用梯度累积（accum_steps=2）在 batch-size=8 下模拟有效 batch-size=16。

指标	值	相比 V2 提升
最佳验证损失	0.000157	2.1×
测试 masked MSE	0.000022	1.3×
相对 L2 均值	0.97%	1.1×
相对 L2 中位数	0.81%	1.1×
相对 L2 P95	1.74%	基本持平

指标	值	相比 V2 提升
相对 L2 最大值	7.14%	1.1×
训练稳定性	全程稳定下降	✓

版本 4 — 学习率预热 + 早停机制 (2024)

在 V3 基础上添加两项训练优化：

- 1. **学习率预热 (Warmup)**：前 5 轮学习率从 0 线性增长到目标值，避免初期大学习率导致的不稳定。
- 2. **早停机制 (Early Stopping)**：验证损失连续 30 轮不下降且训练超过 50 轮时自动停止，防止过拟合和浪费计算资源。

指标	值	相比 V3
最佳验证损失	0.000161	基本持平
测试 masked MSE	0.000022	基本持平
相对 L2 均值	0.98%	基本持平
相对 L2 中位数	0.82%	基本持平
相对 L2 P95	1.87%	基本持平
相对 L2 最大值	7.21%	基本持平
训练轮数	200	patience=50 未触发
训练时间	1h44min	略有缩短

关键改进效果：

- 学习率预热使训练初期更稳定，避免 loss 震荡
- 早停机制可在模型收敛后自动终止，节省计算资源
- 当验证损失在后期出现波动时，早停可防止过拟合

版本 4.1 — 磁轴约束 + 增强求解器 (2026)

在 V4 基础上引入三项改进（工作区未提交版本，对应 data/g_s_fixed_boundary_v41*.npz 系列数据集）：

- 1. **磁轴约束损失**（--axis-weight 0.01，默认启用）：在真实磁轴位置约束 psi_bar = 1
- 2. **求解器增强**：Romberg 网格求解 + 样条插值（任意目标网格）、求解失败样本自动过滤
- 3. **评估增强**：n_cases 指标 + best/worst cases 可视化

运行	数据集	masked MSE	rel L2 均值	rel L2 中位数	rel L2 P95	rel L2 最大	测试样本数
large_v4.1	gs_fixed_boundary_v41.npz (~500)	0.00236	6.17%	1.85%	8.45%	153%	75
v41_fixed	gs_fixed_boundary_v41_clean.npz (~800)	0.00071	3.14%	1.92%	7.19%	78.0%	120

注：P95 与 max 之间差距较大，说明存在少量困难样本（参数域边界附近），与 V1-V4 的现象一致。

运行命令

```
# 基线
python -m gs_pino.train --data data/gs_large.npz --lr 1e-3 \
    --epochs 200 --batch-size 16 --width 64 --modes1 32 --modes2 32 \
    --layers 4 --pde-weight 0.01 --output-dir outputs/large

# 改进版（余弦退火 + 梯度裁剪）
python -m gs_pino.train --data data/gs_large.npz --lr 1e-3 --clip-grad 1.0 \
    --epochs 200 --batch-size 16 --width 64 --modes1 32 --modes2 32 \
    --layers 4 --pde-weight 0.01 --output-dir outputs/large_v2

# 数据翻倍版（梯度累积 + 低 LR）
python -m gs_pino.train --data data/gs_large2k.npz --lr 5e-4 --clip-grad 1.0 \
    --epochs 200 --batch-size 8 --accum-steps 2 --width 64 --modes1 32 --modes2 32 \
    --layers 4 --pde-weight 0.01 --output-dir outputs/large_v3

# 训练优化版（Warmup + 早停）
python -m gs_pino.train --data data/gs_large2k.npz --lr 5e-4 --clip-grad 1.0 \
    --epochs 200 --batch-size 8 --accum-steps 2 --warmup-epochs 5 \
    --patience 50 --min-epochs 50 --width 64 --modes1 32 --modes2 32 \
    --layers 4 --pde-weight 0.01 --output-dir outputs/large_v4
```

11. 自由边界条件扩展（freegs 管线）

11.1 项目概述

自由边界条件扩展使用 **freegs** 库生成数据集，构建从 PF 线圈电流和等离子体参数到极向磁通的 PINO 模型。当前支持两种模型架构（`train_freegs --model`）：

- `planet` — PlaNetCore（Trunk-Branch-Decoder，默认）
- `ufno` — U-FNO v2（12 通道输入，GroupNorm + 残差连接）

最新实验结果（v9，1500 样本）中 PlaNetCore 为最优模型（等离子体区域 rel L2 均值 9.57%），详见第 12 章。

核心区别（相对固定边界管线）：

- **输入无掩码**：网络必须自己学习等离子体-真空界面
- **输出全区域**：预测整个计算域的 `psi_total`（或 `psi_plasma`），而非仅等离子体区域

11.2 自由边界 GS 方程

总极向通量分解为：

$$\begin{aligned}\psi_{\text{total}} &= \psi_{\text{plasma}} + \psi_{\text{coils}} \\ \psi_{\text{coils}} &= \Sigma(I_k \cdot G_k(R, Z))\end{aligned}$$

其中 `psi_coils` 由线圈电流与格林函数解析计算（`freegs compute_psi_coils`）。

11.3 文件结构

```
src/gs_pino/
├─ generate_freegs_dataset.py # 数据集生成（使用 freegs）
├─ data_freegs.py           # 数据集类 FreeBndDataset + build_ufno_input
├─ losses_freegs.py         # 损失函数（GSOperatorLoss、curvature、axis、ip）
├─ train_freegs.py          # 训练 CLI (--model planet/ufno)
├─ evaluate_freegs.py       # 评估 CLI
├─ visualize_freegs.py      # 可视化 CLI
├─ inference_freegs.py      # 推理 CLI（见 14.2）
├─ merge_datasets.py        # 数据集合并工具（见 14.1）
└─ visualize_results.py     # 结果可视化（见 14.3）
```

11.4 输入通道设计（U-FNO v2）

总通道数：12（build_ufno_input）

通道	描述	维度
R_norm	R / 2.0	[1, nx, ny]
Z_norm	Z / 2.0	[1, nx, ny]
Ip_norm	归一化等离子体电流	[1, nx, ny]
paxis_norm	归一化轴压强	[1, nx, ny]
alpha_m_norm	归一化形状参数	[1, nx, ny]
alpha_n_norm	归一化形状参数	[1, nx, ny]
fvac_norm	归一化真空 f 值	[1, nx, ny]
coil0_norm ... coil3_norm	4 个 PF 线圈电流（归一化）	[4, nx, ny]
psi_coils	线圈真空通量场（逐点变化）	[1, nx, ny]

- 说明：
- 9 维标量 measures = [coil0..coil3, Ip, paxis, alpha_m, alpha_n, fvac]，经训练集均值/标准差归一化后广播到全网格
 - ✅ 曾存在通道取值偏移 bug（measures[:, 0:5] 被当作等离子体参数、measures[:, 5:9] 被当作线圈电流，与通道名错位 4 位），已修复为按上述顺序严格取数；data_freegs.py 中 build_ufno_input 带 docstring 注明 measures 顺序约定
 - PlaNetCore 不使用该通道构建，其输入为 (x_meas, R, Z, psi_coils) 四元组（见 12.2）

11.5 参数采样范围

参数	范围
Ip	1.5e5 - 2.5e5 A
paxis	800 - 1500 Pa
alpha_m	1.0 - 2.0
alpha_n	1.5 - 2.5

参数	范围
fvac	1.8 - 2.2
X-points	随机采样目标 X 点位置（freegs 约束求解用）
PF 线圈电流	不采样 ，由 freegs control.constrain(xpoints) 根据目标 X 点自动解出

数据集默认在 65×65 网格求解（freegs 要求 2^n+1 ），FreeBndDataset 预加载时自动插值到 64×64（FFT/AMP 要求 2^n ）。生成器还包含求解重试逻辑与有效性校验（ $\text{psi_total} \approx \text{psi_plasma} + \text{psi_coils}$ ， $\text{rtol } 1\text{e-}3$ ）。

11.6 损失函数组合（PlaNetLoss）

损失类型	权重（CLI 默认）	作用域	描述
mse（interior_mask 加权）	1.0	计算域内部（边缘 10% 缓冲平滑过渡）	数据拟合损失
pde（GSOperatorLoss）	0.01	内部格点	卷积算子计算 $\Delta*\psi$ 与预计算 rhs 的 MSE
curvature	0.5	内部格点	预测场拉普拉斯平方均值（平滑约束）
axis_constraint_loss	0.01	磁轴位置	$\psi_\text{norm} = 1$ 约束（双线性插值）
ip_constraint_loss_freebnd	0.001	等离子体区域	等离子体电流积分约束

GSOperatorLoss 说明：PDE 损失不使用解析形式的 `gs_residual_loss_freebnd`（该函数保留在 `losses_freegs.py` 但未接入训练），而是用 3×3 卷积核（Laplace 核 + Df/dR 核）对预测场计算 Grad-Shafranov 算子 $\Delta*\psi$ ，与数据集预计算的 `rhs` 场做 MSE，并可选 5×5 高斯核平滑（`Gauss_kernel_5x5`）。

curvature_loss（新增）：二阶导平滑约束， $\text{mean}(\Delta\psi^2)$ ，抑制预测场高频振荡。

11.7 训练超参数（train_freegs 默认值）

参数	默认值
epochs	300
batch_size	4（有效 batch = 4 × accum_steps=2 = 8）
lr	5e-4
width / layers / modes1 / modes2	128 / 6 / 32 / 32（U-FNO）
hidden_dim	256（PlaNetCore）
scale_mse / pde / axis / ip / curvature	1.0 / 0.01 / 0.01 / 0.001 / 0.5

参数	默认值
warmup_epochs	10
patience / min_epochs	80 / 150
clip_grad	1.0
amp	True（U-FNO 模式强制关闭：复数 FFT 不支持混合精度）
accum_steps	2
weight_decay	1e-6（AdamW）
其他可选	--dropout、--fourier-freqs（TrunkNet 傅里叶编码）、--use-coil-input（CoilEncoder）、--noise-std（线圈电流噪声注入）、--predict-plasma

11.8 评估指标（evaluate_freegs）

- **Global MSE / Plasma MSE**（psi_total；--predict-plasma 模式额外报告 psi_plasma 与合成 psi_total 两套）
- **Relative L2 Error**（mean/median/P95/max）— 全局与等离子体区域
- 输出：test_metrics.json（checkpoint 同目录）+ test_predictions.pt（键：preds / preds_total / targets_plasma / targets_total / masks / interior_masks / indices）

11.9 完整工作流程

```
# 1. 生成数据集（默认 64 样本；实际实验为 500×3 合并，见 12.3）
python -m gs_pino.generate_freegs_dataset --n-samples 500 --out data/freegs_rhs_500.npz

# 2. 合并多数数据集（可选）
python -m gs_pino.merge_datasets --inputs data/freegs_rhs_500.npz \
    data/freegs_seed100.npz data/freegs_seed200.npz \
    --output data/freegs_merged_1500.npz

# 3. 训练模型（默认 --data data/freegs_merged_1500.npz）
python -m gs_pino.train_freegs --model planet --output-dir outputs/freegs_planet_v9

# 4. 测试验证（无 --output-dir，输出到 checkpoint 所在目录）
python -m gs_pino.evaluate_freegs --checkpoint outputs/freegs_planet_v9/best.pt

# 5. 可视化结果
python -m gs_pino.visualize_freegs --predictions outputs/freegs_planet_v9/test_predictions.pt
```

模块调用需 PYTHONPATH=src 或 pip install -e .（同第 9 章说明）。

11.10 可视化输出（visualize_freegs）

文件	描述
psi_total_comparison_*.png	ψ 分布对比图（真值/预测/误差）
psi_total_error_heatmap_*.png	绝对/相对误差热力图

文件	描述
psi_total_lcfs_comparison_*.png	LCFS 轮廓对比图
psi_total_plasma_zoom_*.png	等离子体区域放大图

--predict-plasma 模式另输出 psi_plasma_* 一套。

12. 当前模型与实验结果（v9 / U-FNO v2）

12.1 版本演进

版本	架构	数据集	等离子体 rel L2 均值	备注
v8	PlaNetCore	1500 样本	~10.06%	历史版本，输出目录已不存在
v9	PlaNetCore + 曲率损失	1500 样本	9.57%	预测 ψ_{total} 的基线
ufno_mse	U-FNO v2（纯 MSE）	1500 样本	10.02%	无物理约束
ufno_v2	U-FNO v2（全物理约束）	1500 样本	18.03%	强物理约束反而变差
plasma	PlaNetCore + predict_plasma	1500 样本	8.64%	预测 ψ_{plasma} （线圈通量解析扣除）
plasma_mse	同 plasma	1500 样本	8.29%	约束 scale 组合调整
plasma_opt	同 plasma	1500 样本	7.55%	权重再优化
plasma_coil	同 plasma + --use-coil-input	1500 样本	6.89%	当前最优 ，线圈通量作为额外输入通道
plasma_soft	同 plasma_coil	1500 样本	未完成	与 plasma_coil 配置一致， 训练中断无评估

12.2 模型架构

12.2.1 PlaNetCore（默认模型）

Trunk-Branch-Decoder 结构：

组件	结构
TrunkNet	R/Z 坐标 2 通道 → Conv2dNornAct (16→32→64→128, 3×3 + LayerNorm + TrainableSwish) + MaxPool → Linear(2048→256→hidden_dim); 可选 FourierEncoding 位置编码 (--fourier-freqs)
BranchNet	9 维参数 → Linear(9→256→512→256→hidden_dim), LayerNorm + TrainableSwish; 可选 dropout (--dropout)
DecoderConv	Branch ⊗ Trunk 逐元素乘 → Linear(hidden→256×4×4) → 4 级 ConvTranspose2d (256→128→64→32→16) + Conv2dNornAct → Conv2d(16→1), 输出 64×64
CoilEncoder (可选, --use-coil-input)	对 psi_coils 场做 4 级卷积编码 (1→16→32→64), 与 Branch 特征拼接后 Linear 投影

- 输入为四元组 (x_meas, R, Z, psi_coils) ; 参数量约 3.2M (hidden_dim=256)
- Conv2dNornAct 模块: Conv2d(3×3) → permute → LayerNorm → TrainableSwish → permute
- TrainableSwish: $x \cdot \text{sigmoid}(\beta \cdot x)$, β 可学习

12.2.2 U-FNO v2 (--model ufno)

固定边界章节 4.5 节所述 UFNO2d_v2 的 12 通道版本: lift → 6×UFNOBlock_v2 (GroupNorm + 残差连接) → proj, width=128、modes=32。AMP 自动关闭。

12.2.3 PlaNetAttention (实验性)

models.py 中定义了带 MultiheadAttention 的变体: branch 特征作 query, trunk 特征作 key/value, 注意力输出与 trunk 相加后进 decoder。通过 --model planet_attn 启用 (train_freegs / evaluate_freegs 均已支持该选项, 输入为四元组 (x_meas, R, Z, psi_coils) , 同 PlaNetCore)。尚未跑过完整实验。

12.3 数据集

- 三个子数据集 (seed 42 / 100 / 200 各 500 样本) 合并为 data/freegs_merged_1500.npz (1500 样本)
- 实际文件: freegs_rhs_500.npz (seed 42)、freegs_seed100.npz 、 freegs_seed200.npz
- 生成脚本保存 greens / psi_plasma_norm / coil_names 字段, 但 merge_datasets.py 合并时不含这些键
- 数据预加载 (_preload_data): 初始化时一次性完成全部插值并缓存, 训练速度提升约 36× (47s/epoch → 1.3s/epoch)
- 划分: 训练 1050 / 验证 225 / 测试 225 (70/15/15, split_indices ; 测试取排列最前, 同固定边界)

12.4 训练策略 (课程学习)

```
epoch 0-9      : warmup, 仅 MSE
epoch 10-99   : 仅 MSE (mse_only_epochs = 100)
epoch 100-249 : PDE 线性 ramp (pde_ramp_epochs = 150, 0 → 满值)
epoch 150+    : axis 约束开启
epoch 200+    : ip 约束开启
epoch 250+    : curvature 约束开启
```

- 学习率: 前 10 epoch 线性预热到 5e-4, 之后余弦退火 (CosineAnnealingLR)
- 优化器 AdamW (weight_decay=1e-6), 梯度裁剪 1.0
- 早停: patience=80, min_epochs=150
- history.json 记录每轮 train_/val_ 的 rmse / pde / axis / ip / curvature / normalized_error / total 及各约束的 scale

12.5 评估结果（225 测试样本）

12.5.1 v9（PlaNetCore + curvature）

最优 val_total = 3.86e-05（epoch 30），共训练 110 epochs（早停终止）。

指标	全局	等离子体区域
MSE	3.91e-05	9.56e-05
rel L2 mean	12.25%	9.57%
rel L2 median	8.96%	6.46%
rel L2 P95	29.85%	27.43%
rel L2 max	69.7%	48.8%

12.5.2 ufno_mse（U-FNO v2 纯 MSE，scale 全部为 0）

最优 val_total = 4.33e-05（epoch 226），共训练 326 epochs。

指标	全局	等离子体区域
MSE	4.94e-05	1.28e-04
rel L2 mean	13.68%	10.02%
rel L2 median	10.52%	7.03%

12.5.3 ufno_v2（U-FNO v2 全物理约束）

最优 val_total = 1.74e-04（epoch 30），共训练 201 epochs。

指标	全局	等离子体区域
MSE	1.79e-04	2.47e-04
rel L2 mean	30.06%	18.03%

12.5.4 predict_plasma 系列（当前最优）

--predict-plasma 模式下模型直接预测 ψ_{plasma} （线圈真空通量由 Green 函数解析计算， $\psi_{\text{total}} = \psi_{\text{plasma}} + \psi_{\text{coils}}$ ）。测试集指标（225 样本）：

版本	等离子体 rel L2	等离子体 MSE	全局 MSE	ψ_{total} rel L2	说明
plasma	8.64%	1.09e-04	3.82e-05	10.57%	首个 predict_plasma 实验
plasma_mse	8.29%	1.14e-04	3.81e-05	10.19%	约束 scale 调整
plasma_opt	7.55%	9.68e-05	3.15e-05	9.28%	权重再优化
plasma_coil	6.89%	1.02e-05	1.61e-05	8.33%	+ --use-coil-input

关键发现：

- **plasma_coil 为当前全局最优**（等离子体 rel L2 6.89%），比 v9（9.57%）显著改善；psi_coils 场作为第 12 个输入通道（CoilEncoder 编码）信息量最大
- 预测 ψ_{plasma} 天然剔除线圈主导成分（线圈通量约占总通量 91.8%），让网络专注于等离子体形状
- 通道错位 bug（见 11.4 节）修复后数据才真正按 [coil0-3, Ip, paxis, alpha_m, alpha_n, fvac] 顺序输入，coil 系列结果有效
- ufno_v2（强物理约束）显著变差——U-FNO v2 下物理约束的加入方式仍需调优（可能与 AMP 强制关闭、课程学习时间表不匹配有关）
- 少数样本误差大（P95 偏高），多位于参数域边界

12.6 运行命令

```
# v9 训练 (PlaNetCore + 曲率损失)
python -m gs_pino.train_freegs --model planet --data data/freegs_merged_1500.npz \
    --output-dir outputs/freegs_planet_v9 --hidden-dim 256 \
    --scale-mse 1.0 --scale-pde 0.01 --scale-axis 0.01 \
    --scale-ip 0.001 --scale-curvature 0.5

# U-FNO 训练 (纯 MSE 对照)
python -m gs_pino.train_freegs --model ufno --data data/freegs_merged_1500.npz \
    --output-dir outputs/freegs_ufno_mse \
    --scale-pde 0.0 --scale-axis 0.0 --scale-ip 0.0 --scale-curvature 0.0

# 评估 (无 --output-dir, 输出到 checkpoint 目录)
python -m gs_pino.evaluate_freegs --checkpoint outputs/freegs_planet_v9/best.pt

# 可视化
python -m gs_pino.visualize_freegs --predictions outputs/freegs_planet_v9/test_predictions.pt
```

12.7 关键改进与经验总结

改进项	说明
数据预加载	插值操作 (RegularGridInterpolator) 从 __getitem__ 移到初始化阶段一次性缓存，训练速度提升约 36×
课程学习	先 MSE-only 再逐步引入 axis/ip/PDE/curvature，避免物理约束突然引入导致训练不稳定
曲率损失	二阶导平滑约束 (scale 0.5) 在 v9 中进一步压低等离子体区域误差
损失权重	PDE=0.01 / axis=0.01 / ip=0.001 / curvature=0.5 为当前较优配置；U-FNO 上过强物理约束 (ufno_v2) 反而恶化精度
归一化指标	ψ_{total} 绝对值很小 (均值约 0.03)，RMSE 与归一化误差比 MSE 更直观
网格尺寸	freegs 要求 2^{n+1} (65)，FFT/AMP 要求 2^n (64)，训练时插值到 64×64

13. 解析解验证 (verification/)

src/gs_pino/verification/ 目录包含一组**广义 Solov'ev 解析解**与 gspack 数值求解器的交叉验证脚本（均未提交 git，独立可运行，输出 PNG 到 outputs/）：

文件	功能
verify_solovev.py / _2d / _analytic / _math / _correct	纯解析验证：广义 Solov'ev 解是否满足 GS 方程 ($\Delta^*\psi = -\mu_0 R J_\phi$)，1D 归一化坐标与 65×65 二维网格两种形式
verify_solovev_canonical.py / _gspack / _final	gspack 参数化（Jtor 公式 + gs_sparse_2nd 算子）下解析解与 gspack 数值解对比
verify_gspack_consistency.py / _gs	gspack 解自治性 ($A @ \psi + \mu_0 R J_\phi$ 残差) + 网格收敛测试 (33/65/129 三档)
benchmark_solovev.py	主基准 CLI：解析 Solov'ev vs gspack vs freegs（可选），输出 RMSE / max error / rel L2 / R_axis 差等 4 图（参数：--R0 --a --kappa --delta --lp --betap --alpha-m --alpha-n --nx/--ny --gspack-compatible）
benchmark_summary.py	汇总验证（固定 R0=1, a=0.5, kappa=1, $\delta=0$, lp=2e5, betap=0.5, am=1, an=2, 65×65）

验证结论：

- 1. gspack 固定边界求解器正确求解 GS 方程（自治残差内域 RMSE PASSED）
- 2. 广义 Solov'ev 平衡是有效的 GS 方程解析解
- 3. Solov'ev 与 gspack 数值解之间的差异可由 Shafranov 位移和边界效应合理解释
- 4. 建议使用 gspack 数值解作为固定边界平衡验证的基准

注意：gspack 相关脚本硬编码 d:/D_F/Fusion/AI/PINN/gspack2_TRAE 路径（sys.path 注入）。verify_solovev_gspack.py 的输出路径已修复为基于仓库根目录的 outputs/（不再依赖 CWD），可直接从任何目录运行（PYTHONPATH=src python src/gs_pino/verification/verify_solovev_gspack.py）。

14. 辅助工具与脚本

14.1 merge_datasets.py

合并多个 freegs 数据集：--inputs f1.npz f2.npz ... --output out.npz，沿样本轴拼接 R, Z, psi_total, psi_plasma, psi_coils, mask, rhs, coil_currents, params, axes, L, Beta0。

14.2 inference_freegs.py

自由边界推理 CLI：加载 checkpoint 后由 compute_greens / compute_psi_coils 计算线圈真空通量，输出 psi_total = psi_plasma + psi_coils。已修复：导入改为 data_freegs.build_ufno_input + models.PlaNetCore/UFNO2d_v2，按 checkpoint 读取 model_type / hidden_dim / dropout / fourier_freqs / use_coil_input / modes / width / layers / predict_plasma 重建模型，输入统一转为 float32（conv 权重为 float32）。支持 --model planet|ufno 两种 checkpoint。

14.3 visualize_results.py

从 checkpoint 读取 test_predictions.pt 生成等值线对比 / 误差分布 / R-Z 线切割 / 散点图 / 误差直方图。CLI 为 --checkpoint --data --num-samples（没有 --predictions / --output-dir）。已修复键兼容：优先读 preds_total / targets_total（ ψ_{total} 对比），旧格式 preds / targets 自动回退，两种 evaluate 输出均可渲染。

14.4 configs/（供参考，代码不加载）

- `default.yaml` — 小型开发配置（64 样本 64×64 ，20 epochs，modes 16/16 width 32）
- `practical.yaml` — 实用配置（4096 样本 128×128 ，250 epochs，modes 24/24 width 64 layers 5，pde 0.02 / bc 0.10）

两文件为参数备忘（全代码无 YAML 解析逻辑），同名参数由 `scripts/` 内 shell 脚本硬编码镜像。

14.5 scripts/

- `run_smoke.sh` — 冒烟测试：16 样本 32×32 数据集 → 训练 2 epochs 迷你模型 → 评估 3 张图
- `run_practical.sh` — 端到端实用流程：4096 样本 128×128 （seed 2026）→ 训练 250 epochs → 评估 24 图；`export PYTHONPATH=...:src` 后调用 `python -m gs_pino.*`；环境变量可覆盖 `DATA_PATH / RUN_DIR / EVAL_DIR / N_SAMPLES / NR / NZ / EPOCHS / BATCH_SIZE`

15. 依赖

Python 包

- `torch >= 2.0`
- `numpy`
- `matplotlib`
- `tqdm`

外部求解器 (可选)

- [GS_solver](#) — 经典有限差分 GS 求解器，需克隆至与 `GS_PINO` 同级目录（默认目录名为 `gspack2_TRAE`）
- [freegs](#) — 自由边界 GS 求解器，用于生成自由边界数据集